



AGENTIC AI · EXPLAINED LIKE YOU'RE NEW

How Agentic AI Actually Works



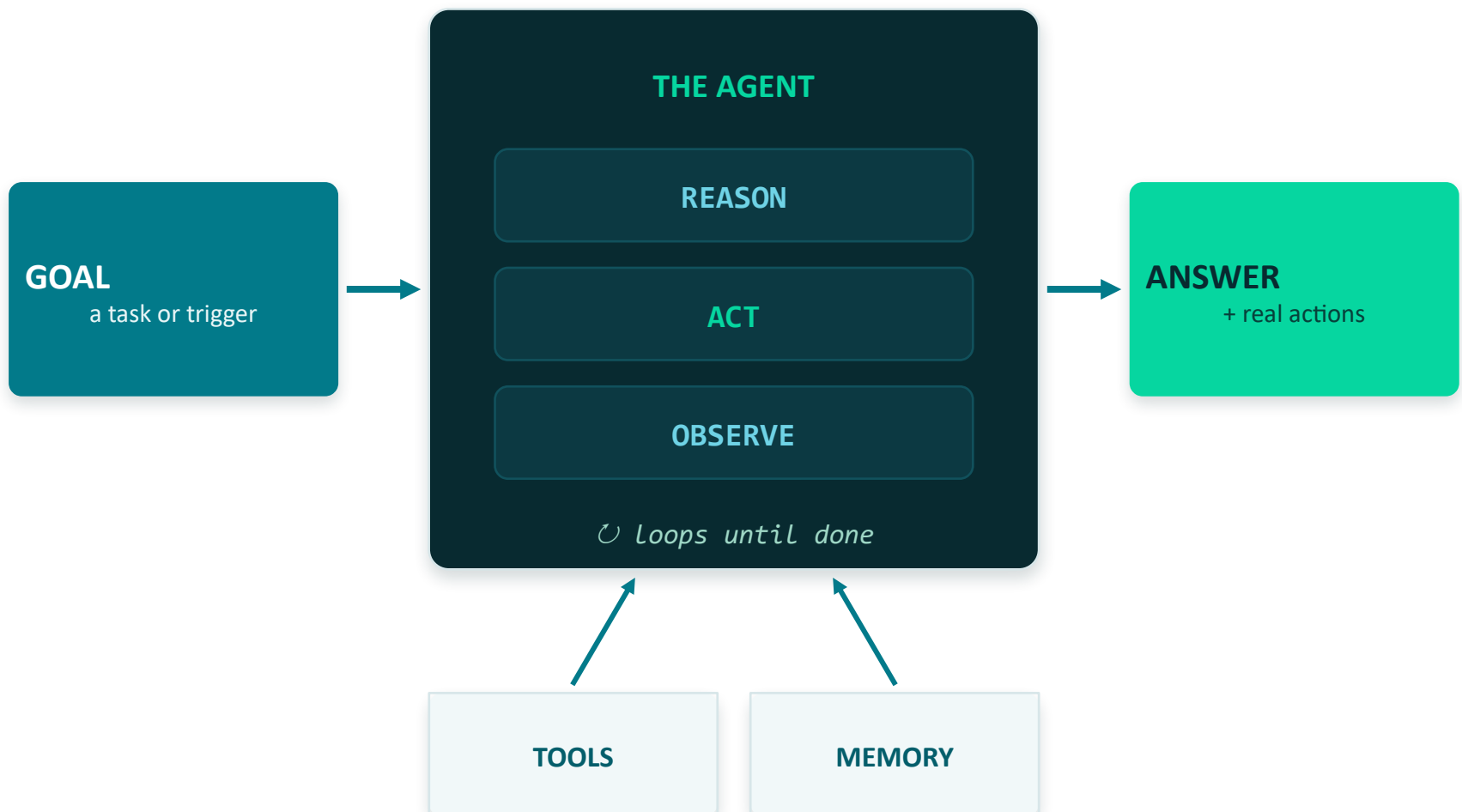
intro.sh

```
$ ask-ai "teach me agentic AI, from zero"  
> no buzzwords. real examples. one step at a time.  
> if you can read code, you can get this.
```

Swipe to learn →

The whole idea in one picture

Everything in this deck is just zooming into the middle box.



A goal goes in. The agent thinks, uses tools, checks the result, repeats — then the answer comes out.

What is an LLM, really?

Think of a friend who has read the entire internet — but can only talk. That's a Large Language Model.



It does ONE thing: guess the next word — over and over, really well.

That's the whole trick. No magic. It's a very, very good autocomplete.



llm.txt

```
prompt "The capital of Japan is"
```

```
the model asks itself, again and again:  
  what word most likely comes next?
```

```
→ "Tokyo" (then ".", then stop)
```

```
output "Tokyo."
```

Predict a word, add it, predict the next. Do that fast enough and it looks like thinking.

THE CATCH

On its own, the brain is stuck in a room

Brilliant, but locked in a box: no phone, no internet, no notepad. By itself an LLM can only do this:



Can't DO anything

It only produces text. It can't send an email, query a database, or open a webpage.



Forgets instantly

Once the chat ends, it remembers nothing. Every conversation starts from a blank slate.



Can be confidently wrong

It predicts likely words — not verified facts. It will state a wrong answer with a straight face.

So how do we turn this talking brain into something that gets real work done? →

Give the brain hands: tools

A tool is just a function the model is allowed to call. The model asks; your code runs it and hands back the result.

```
tool-call.json

user  "What's the weather in Pune?"

// the LLM can't check weather – so it asks for a tool
model → {
  "tool": "get_weather",
  "args": { "city": "Pune" }
}

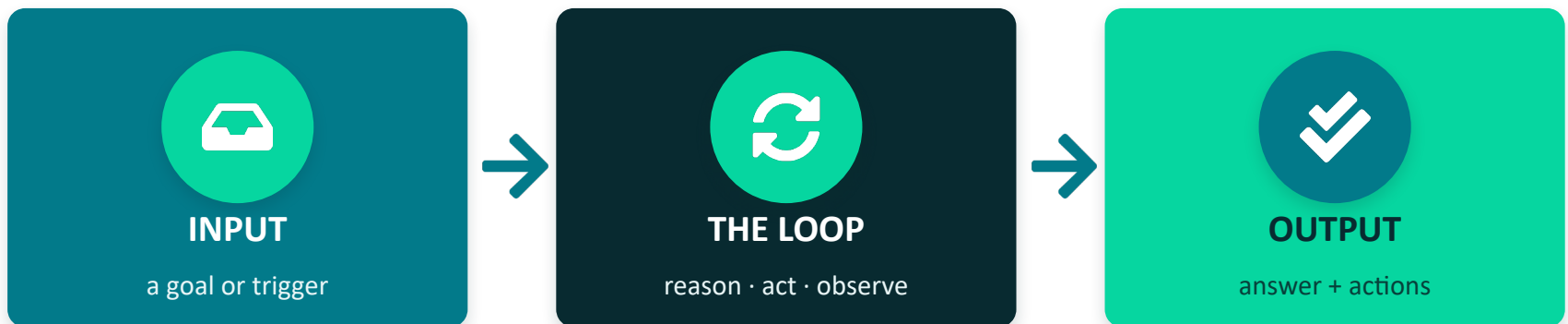
// your code runs it, returns real data
tool  → { "tempC": 31, "sky": "clear" }

model → "It's 31°C and clear in Pune."
```

A brain that can call tools and react to the results = an AGENT. That's the whole leap.

Forget the jargon — it's one loop

Every “agentic” system has the same shape: a goal goes in, the model decides → acts → looks, again and again, until done.



```
loop.py

while not done:
    step = model.reason(state)    # think
    result = run_tool(step)      # act
    state = state + result        # observe
```

The trick: it never answers in one shot. It takes a step, reads what came back, then picks the next move.

Reason → Act → Observe

This is called ReAct. One turn at a time — here's what each word means.



REASON

The model thinks out loud: “what should I do next to get closer to the goal?”



ACT

It picks one tool and calls it — search, query a DB, run code, send a request.



OBSERVE

It reads the result of that action — then feeds it back in and loops.



Repeat until the goal is met — each observation shapes the next decision.

“Book a table for 2, Friday 7pm”

Follow the loop turn by turn. Notice it stops to ask you before doing something it can't undo.

```
agent.log

GOAL  book a table for 2, Friday 7pm
— turn 1 —
reason  need a place; recall: home=downtown, hates loud
act     maps_search("dinner, downtown")
observe 3 options found
— turn 2 —
reason  pick the quiet, top-rated one
act     check_availability(Fri 19:00)
observe 7:00 full · 7:30 open
— turn 3 ⚠ ask the human —
reason  time changed + booking is irreversible
act     ask_user("7:30 ok?") → "yes"
— turn 4 —
act     book_table(Fri 19:30) → #A12
done ✓ added to calendar · loop exits
```


What's really in the box: 6 layers



1 • Input & guardrails

Auth, validation, PII scrub, prompt-injection checks — before anything runs.



2 • Agentic core

The planner + the reason-act-observe loop. The decision-maker.



3 • Tools / actions

APIs, DB queries, code, search — how the agent touches the real world.



4 • Memory & state

The context window, a vector store (RAG), and checkpoints to pause/resume.



5 • Output gates

Evaluator (is it correct?) + guardrails (is it safe?) + optional human OK.



6 • Observability

Tracing, logging, evals, cost/token tracking, retries — wraps everything.

The next few slides zoom into the parts people get confused about.

Three kinds of “memory”

People lump these together and get confused. They're three different things.



Short-term

like a sticky note

“What did we just say?”

The context window. This run only — gone when it ends.



Long-term

like a filing cabinet

“What do I know?”

A vector store (RAG). Facts that survive across runs.



Workflow state

like a bookmark

“Where was I?”

Checkpoints. Lets a long job pause and resume later.

Sticky note = now. Filing cabinet = forever. Bookmark = where you paused.

“Is it good?” vs “Is it safe?”

A result can pass one and fail the other. Keep them separate — they answer different questions.



Evaluator

“Is it correct / good enough?”

- Judges the quality of the answer
- Can trigger a retry / redo
- Often the model grading itself
- Pattern: evaluator-optimizer



Guardrails

“Is it safe, allowed, well-formed?”

- Checks rules + policy + safety
- Blocks leaks (e.g. an SSN)
- A fixed rule, not an opinion
- Runs on BOTH input and output

Evaluator = a picky teacher grading the work. Guardrails = a security guard at the door.

Workflow vs Agent



Workflow

YOU fix the path

- Steps decided ahead of time
- The model fills in the blanks
- Predictable, cheap, easy to debug
- Most “AI products” live here



Agent

the MODEL picks the path

- The model decides its own steps
- Reacts + self-corrects at runtime
- Flexible, but harder to control
- Use only when it can't be scripted

*The one question that settles every case:
who decided the steps — you (ahead of time) or the model (at runtime)?*

5 patterns you'll see everywhere

Almost every “agentic” product is one of these five — or a mix of them.



Prompt chaining

Output of step 1 feeds step 2, in a fixed order.

WORKFLOW



Routing

Sort the input, send it down one of several fixed paths.

WORKFLOW



Parallelization

Run several calls at once, then combine the results.

WORKFLOW



Orchestrator-workers

A lead splits a task into subtasks chosen at runtime.

AGENT



Evaluator-optimizer

Generate → grade → retry until it's good enough.

AGENT

First three are workflows. Only the last two hand real decisions to the model.

Who's the boss? Orchestrator vs Worker

When one agent isn't enough, you split the roles: a coordinator and the doers it manages.



Orchestrator

the coordinator

- Reads the goal, makes a plan
- Splits it into subtasks
- Assigns each to a worker
- Judges + assembles the result



Worker

the doer

- Gets one assigned subtask
- Uses tools to finish it
- Returns the result up
- Revises if asked

Like a team lead and their team: one plans and reviews, the others execute.

Workflow first. Agent only if you must.

Start with the simplest thing that works. Add freedom only when the task truly needs it.

Can you write the steps in advance?

→ use a WORKFLOW. Done.

↓ if no

Are the steps the same every time?

→ chaining or routing.

↓ if no

Do the steps depend on what's found at runtime?

→ NOW reach for an AGENT.

Agents are powerful but pricier and harder to predict. Don't reach for one out of habit.

Every loop costs tokens — and money

Tokens are the chunks of text the model reads and writes. You pay per token. Loops repeat → cost adds up fast.

```
cost.txt

"agentic" ≈ 1 token      "un-be-liev-able" ≈ 4 tokens

1 loop = a few hundred tokens
20 loops = thousands of tokens → watch the bill
```



Cap the loops — set a max number of turns so it can't run forever



Trim the context — don't resend the whole history every single call



Right-size the model — use a small model for easy steps, big for hard ones

5 traps beginners fall into



Using an agent when a workflow would do

→ more cost, more bugs, no benefit



No loop limit

→ it spins forever, burning tokens



Trusting the output blindly

→ no evaluator, no guardrails = surprises



Giving it too many tools

→ it gets confused and picks the wrong one



No tracing

→ when it breaks, you can't see why

Every one of these comes down to the same fix: start simple, watch it, add limits.

Build your first agent: a checklist

Six small steps. Do them in order and you'll have a working, safe little agent.

first-agent.md

- [1] Pick ONE clear goal
 “summarize my unread emails”
- [2] Give it 1-3 tools (not 20)
 list_emails(), read_email()
- [3] Write the reason-act-observe loop
 with a max-turns cap
- [4] Add a guardrail + a human OK
 for anything risky
- [5] Log every step (tracing)
- [6] Run, read the trace, improve – repeat

Every term, in one line

Screenshot this one. It's the whole deck, compressed.

LLM

a brain that predicts the next word

Token

a chunk of text; you pay per token

Tool

a function the model can call

Agent

an LLM that loops with tools

ReAct

reason → act → observe, repeat

Context

the short-term memory of one run

RAG

look up real docs, then answer

Guardrail

a safety rule on in/out

Evaluator

checks if the answer is good

Workflow

steps you fixed in advance

Orchestrator

the agent that splits + assigns

Observability

tracing, tokens, cost, evals



YOU MADE IT

You can now explain agentic AI.

The takeaway: agents are more capable but less predictable. So reach for the simplest thing that works — a workflow first, an agent only when the task genuinely can't be pre-scripted.

Found this useful?

I'm a developer learning AI engineering in public. Follow + repost so the next person can find it — and tell me what to break down next.